Perulangan/Iterasi (Looping)

`for` loop dan `while` loop

Andreas Syaloom Kurniawan

Faculty of Economics and Business, Universitas Gadjah Mada

Kenapa kita butuh loop?

Data: bilangan 2, 4, 6, 8, 10

Masalah: menghitung kuadrat setiap bilangan.

Dicari: $2^2, 4^2, \ldots, 10^2$.

Cara yang sudah kita bisa:

```
x1 = 2^2;
x2 = 4^2;
x3 = 6^2;
x4 = 8^2;
x5 = 10^2;
```

Perhatikan polanya:

- Perhitungannya selalu sama: bilangan dipangkatkan 2.
- Yang berubah hanya bilangannya.
- Untuk 1.000 bilangan, cara ini butuh 1.000 baris.

Ide loop: tulis perhitungannya **sekali**, lalu minta MATLAB mengulangnya untuk setiap bilangan.

Tiga istilah dasar

Istilah	Arti
loop	Struktur untuk mengulang suatu perhitungan (atau beberapa perhitungan) berkali-kali.
pass	Satu kali pengulangan. Loop yang berulang 5 kali punya 5 pass. Sering juga disebut *iterasi*.
kondisi berhenti	Aturan yang menentukan kapan loop selesai.

Analogi: naik tangga. Setiap anak tangga yang dinaiki adalah satu pass. Gerakannya selalu sama: angkat kaki, naik satu anak tangga. Kita berhenti ketika sampai di lantai tujuan: itulah kondisi berhenti.

Dua cara menentukan kapan berhenti

`for`: jumlah pass sudah diketahui

Seperti mengabsen 30 mahasiswa satu per satu. Sebelum mulai, kita sudah tahu akan ada tepat 30 panggilan.

Contoh di ekonomi: simulasi model selama $T = 100$ periode.

`while`: ulangi selama kondisi benar

Seperti mengaduk kopi *selama* gulanya belum larut. Kita tidak tahu berapa kali harus mengaduk; kita hanya terus mengecek.

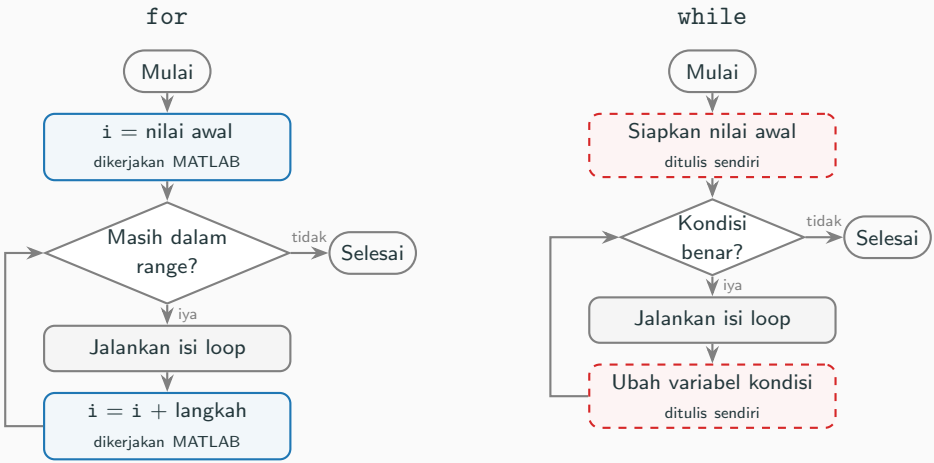
Contoh di ekonomi: iterasi mencari steady state sampai konvergen.

Pertanyaan kunci sebelum menulis loop:

"Apakah saya sudah tahu berapa kali harus mengulang?"

Ya $\rightarrow$ `for` Tidak $\rightarrow$ `while`

Alurnya sama, yang berbeda siapa yang mengurus



Kotak biru dikerjakan MATLAB secara otomatis. **Kotak merah putus-putus** harus kamu tulis sendiri.

`for` Loop

Bahan dasar: range `m:s:n`

Sebelum masuk ke loop, kenali dulu cara MATLAB membuat deretan angka.

```
>> 2:2:10
ans =
     2     4     6     8    10
```

- `m` nilai awal (di sini 2)
- `s` langkah, ditambahkan tiap kali (2)
- `n` nilai akhir, batas atas (10)

Ditulis	Hasil	Catatan
`1:5`	`1 2 3 4 5`	langkah dihilangkan = 1
`1:3:10`	`1 4 7 10`	10 tepat terkena
`0:5:12`	`0 5 10`	15 sudah melewati 12
`10:-2:2`	`10 8 6 4 2`	langkah negatif: mundur
`2:2:5`	`2 4`	berhenti sebelum melewati 5
`10:2:2`	*kosong*	awal sudah melewati batas

`for` loop menjalankan satu pass untuk setiap angka dalam range. Jika isi range bisa ditebak, jumlah pass juga sudah diketahui.

Sintaks `for` dan contoh pertama

```
for variable = m:s:n
    statement(s)
end
```

Dibaca: "**untuk setiap** nilai `variable` di dalam `m:s:n`, jalankan `statement(s)`."

- `variable` disebut loop index; namanya bebas (`i`, `k`, `tahun`).
- Di setiap pass, MATLAB otomatis mengisinya dengan angka berikutnya dari range.
- `end` menandai akhir bagian yang diulang.

```
for i = 1:3
    disp(i)
end
```

```
     1
     2
     3
```

Pass	`i`	yang terjadi
1	1	tampilkan 1
2	2	tampilkan 2
3	3	tampilkan 3

Range `1:3` berisi 3 angka, jadi ada 3 pass.

Menampilkan hasil: `disp`, bukan `print`

```
>> x = [4 16 36];
>> x                  % tanpa titik koma
x =
     4    16    36
>> disp(x)            % nilainya saja
     4    16    36
>> fprintf('%d ', x) % format sendiri
4 16 36
```

- Di MATLAB, `print` dipakai untuk mencetak atau menyimpan figure (grafik), bukan menampilkan isi variabel. Itu berbeda dengan `print` di Python.
- Baris yang diakhiri `;` tetap dijalankan, tetapi hasilnya tidak ditampilkan.
- Di dalam loop, hilangkan `;` atau pakai `disp` jika ingin melihat nilai di setiap pass.

Contoh 1: kuadrat, tetapi hasilnya tertimpa

Data: range `2:2:10`

Masalah: menghitung kuadrat setiap angka dengan loop.

Dicari: vektor berisi $4, 16, 36, 64, 100$.

```
for i = 2:2:10
    x = i^2;
end
x
```

```
x =
   100
```

Pass	`i`	isi `x` setelah pass
1	2	4
2	4	16
3	6	36
4	8	64
5	10	100

`x` hanya menampung satu angka. Setiap pass menimpa hasil pass sebelumnya, sehingga yang tersisa hanya hasil pass terakhir.

Contoh 1 (perbaikan): simpan hasil di posisinya sendiri

```
k = 0;       % posisi
for i = 2:2:10
    k = k + 1;
    x(k) = i^2;
end
x
```

```
x =
     4     16     36     64    100
```

Pass	i	k	isi x setelah pass
1	2	1	[4]
2	4	2	[4 16]
3	6	3	[4 16 36]
4	8	4	[4 16 36 64]
5	10	5	[4 16 36 64 100]

- `k` naik 1 di setiap pass, sehingga tiap hasil masuk ke posisi baru: 1, 2, 3, 4, 5.
- Kenapa bukan `x(i)`? `i` bernilai 2, 4, 6, ... Posisi 1, 3, 5, ... akan terisi 0 dan panjang `x` menjadi 10, bukan 5.

Versi tanpa loop, dan kapan loop tetap dibutuhkan

Untuk perhitungan sederhana, MATLAB bisa menghitung satu vektor sekaligus:

```
>> x = (2:2:10).^2
x =
     4    16    36    64   100
```

Titik sebelum ^ berarti operasi per elemen. Hasilnya sama dengan loop tadi dan biasanya lebih cepat.

Loop tetap dibutuhkan ketika sebuah pass bergantung pada hasil pass sebelumnya, misalnya simulasi AR(1):

$$y_t = 0.9\, y_{t-1} + \varepsilon_t$$

```
T = 100;
y = zeros(T,1);        % preallocate
e = randn(T,1);
for t = 2:T
    y(t) = 0.9*y(t-1) + e(t);
end
```

Jebakan 1: `n` adalah batas, bukan nilai terakhir

```
for i = 2:2:5
    disp(i)
end
```

```
     2
     4
```

Angka 6 sudah melewati 5, jadi loop berhenti setelah 2 pass. `i` tidak pernah bernilai 5.

```
for i = 10:2:2
    disp(i)
end
```

```
     (tidak ada output)
```

Range-nya kosong, jadi 0 pass: isi loop tidak pernah dijalankan dan tidak ada error. Untuk menghitung mundur, langkahnya harus negatif: `10:-2:2`.

Jebakan 2: mengubah loop index di dalam loop

```
for mama = 3:2:100
    mama = mama - 5
end
```

Pass	`mama` di awal	yang tercetak
1	3	-2
2	5	0
3	7	2
⋮	⋮	⋮
49	99	94

Kodenya jalan, tetapi:

- Perubahan `mama - 5` hanya berlaku sampai akhir pass. Pass berikutnya tetap mengambil angka selanjutnya dari range.
- Jumlah pass tetap 49. Setelah loop selesai, `mama` bernilai 94.
- Editor MATLAB memberi peringatan oranye: *loop index is changed inside of a FOR loop*.

Kebiasaan yang aman: biarkan loop index hanya menghitung pass, lalu simpan hasil ke variabel lain, misalnya `y = mama - 5;`.

Latihan `for`

Kerjakan dulu di kertas, lalu cek di MATLAB. Jawaban ada di akhir.

1. Berapa jumlah pass pada `for i = 1:3:10`?
2. Apa isi `x` setelah kode berikut?

```
for i = 5:-1:3
    x = i;
end
```

3. Berapa nilai `k` setelah loop `2:2:10` pada contoh perbaikan selesai?
4. Tulis loop untuk menyimpan $1^3, 2^3, \ldots, 6^3$ ke dalam vektor `y`.
5. Ubah contoh AR(1) agar menyimpan y_t untuk koefisien 0.5, lalu bandingkan `y(100)` kedua versi dengan `e` yang sama.

`while` Loop

Bahan dasar: kondisi, lalu sintaks `while`

`while` bergantung pada kondisi: pernyataan yang jawabannya benar (`1`) atau salah (`0`).

```
>> 5 <= 10
ans =
  logical
   1
```

Sebaliknya, `12 <= 10` menghasilkan `0`.

```
while kondisi
    statement(s)
end
```

Dibaca: "**selama** kondisi masih benar, jalankan `statement(s)`." Berbeda dengan `for`, ada tiga hal yang kamu urus sendiri:

1. **Siapkan**: beri nilai awal sebelum loop.
2. **Cek**: kondisi dicek *sebelum* setiap pass.
3. **Ubah**: ubah variabel kondisi di dalam loop; tanpa ini loop tidak pernah berhenti.

Contoh 2: kuadrat yang sama, ditulis dengan `while`

```
i = 2;                  % siapkan
k = 0;
while i <= 10           % cek
    k = k + 1;
    x(k) = i^2;
    i = i + 2;          % ubah
end
x
```

```
x =
     4     16     36     64    100
```

Pass	`i`	`i <= 10?`	`x(k)`	`i` baru
1	2	benar	4	4
2	4	benar	16	6
3	6	benar	36	8
4	8	benar	64	10
5	10	benar	100	12
–	12	salah	berhenti	

Hasilnya sama dengan versi `for`. Bedanya, `i = i + 2` ditulis sendiri, sedangkan di `for` MATLAB yang mengerjakannya.

Jebakan: infinite loop dan 0 pass

Baris “ubah” terlupa

```
i = 2;
while i <= 10
    x = i^2;
    % i = i + 2;    (terlupa)
end
```

Di setiap pass `i` tetap 2, sehingga `2 <= 10` selalu benar dan loop tidak pernah berhenti: infinite loop. Hentikan dengan **Ctrl+C** di Command Window.

Kondisi sudah salah sejak awal

```
i = 12;
while i <= 10
    x = i^2;
    i = i + 2;
end
x
```

```
Unrecognized function or variable 'x'.
```

Kondisi dicek sebelum pass pertama. Karena `12 <= 10` salah, isi loop tidak pernah dijalankan (0 pass), sehingga `x` tidak pernah dibuat.

Contoh 3: iterasi sampai konvergen

Data: tebakan awal $x_0 = 1$, rumus perbaikan $x_{\text{baru}} = \frac{1}{2}\left(x + \frac{2}{x}\right)$

Masalah: memperbaiki tebakan berulang kali sampai perubahannya sangat kecil.

Dicari: $\sqrt{2}$, dengan toleransi 10^{-8}.

```
x = 1;
err = 1;
while err > 1e-8
    x_baru = 0.5*(x + 2/x);
    err = abs(x_baru - x);
    x = x_baru;
end
x
```

Pass	x	err	err > 1e-8?
1	1.500000	0.5	benar
2	1.416667	0.0833	benar
3	1.414216	0.00245	benar
4	1.414214	2.12×10^{-6}	benar
5	1.414214	1.59×10^{-12}	salah

- Tidak ada loop index; yang menghentikan loop adalah `err` yang makin kecil. Jumlah pass (5) baru diketahui setelah selesai.
- Pola yang sama dipakai untuk mencari steady state atau value function iteration.

Pengaman: batas iterasi dan `break`

Batas iterasi

Jika belum tentu konvergen, tambahkan penghitung agar loop pasti berhenti.

```
iter = 0;
while err > 1e-8 && iter < 1000
    % ... isi loop ...
    iter = iter + 1;
end
if iter == 1000
    warning('Belum konvergen')
end
```

`break`

Menghentikan loop saat itu juga; berlaku untuk `for` maupun `while`.

```
for i = 1:100
    if i^2 > 50
        break
    end
    disp(i)
end
```

Menampilkan 1 sampai 7, lalu berhenti karena $8^2 = 64 > 50$.

Latihan `while`

Kerjakan dulu di kertas, lalu cek di MATLAB. Jawaban ada di akhir. Koma memisahkan perintah dalam satu baris.

1. Berapa nilai akhir `i` dan jumlah pass?

```
i = 1;  while i < 20,  i = i*3;  end
```

2. Berapa nilai akhir `i`?

```
i = 10;  while i > 0,  i = i - 3;  end
```

3. Berapa pass yang dijalankan?

```
i = 5;  while i < 5,  i = i + 1;  end
```

4. Apa masalahnya, dan bagaimana memperbaikinya?

```
n = 1;  while n < 10,  disp(n),  end
```

Apa Saja yang Bisa Diulang

Isi loop tidak terbatas pada satu rumus (1)

Apa pun yang ditulis di antara baris loop dan end dijalankan ulang di setiap pass.

Indeks atau posisi

```
for k = 1:5
    x(k) = 2*k;
end
% x = [2 4 6 8 10]
```

Variabel yang diperbarui

```
total = 0;
for i = 1:5
    total = total + i;
end
% total = 15
```

Function

```
f = @(t) 3*t + 1;
for i = 1:3
    y(i) = sqrt(i) + f(i);
end
```

Indeks: hasil tiap pass masuk ke posisi berbeda. **Variabel yang diperbarui**: nilai baru dihitung dari nilai lama (akumulasi). **Function**: function bawaan maupun buatan sendiri bisa dipanggil di setiap pass.

Isi loop tidak terbatas pada satu rumus (2)

Kondisi di dalam loop

```
for i = 2:2:6
    if i > 4
        x(i/2) = i^2;
    else
        x(i/2) = i^2 - 1;
    end
end
% x = [3 15 36]
```

Loop di dalam loop

```
for r = 1:3
    for c = 1:3
        A(r,c) = r*c;
    end
end
% tabel perkalian 3x3
```

Daftar nilai bebas

```
for v = [3 7 1]
    disp(v*10)
end
for kota = ["Jogja" "Solo"]
    disp(kota)
end
```

Loop juga dipakai untuk membaca banyak file, membuat banyak grafik, atau menjalankan simulasi dengan parameter berbeda-beda.

Ringkasan

	`for`	`while`
Dipakai ketika	jumlah pass sudah diketahui	jumlah pass belum diketahui
Berhenti karena	angka di range habis	kondisi menjadi salah
Yang memperbarui	MATLAB, otomatis	kamu, di dalam loop
Risiko utama	range salah tulis (0 pass)	infinite loop
Contoh ekonomi	simulasi $T = 100$ periode	iterasi sampai konvergen

- Simpan hasil tiap pass di posisinya sendiri (`x(k)`), jika tidak hasilnya tertimpa.
- Jangan mengubah loop index `for` di dalam loop.
- Di `while`: siapkan, cek, ubah; tambahkan batas iterasi jika belum tentu konvergen.

Jawaban latihan `for`

1. 4 pass: `i` bernilai 1, 4, 7, 10.
2. `x = 3`. Loop menghitung mundur 5, 4, 3; karena `x` tanpa indeks, hanya nilai pass terakhir yang tersisa.
3. `k = 5`, sama dengan jumlah pass.
4. Salah satu jawaban (di sini `y(i)` aman karena `i` $= 1, 2, 3, \ldots$):

```
for i = 1:6
    y(i) = i^3;     % [1 8 27 64 125 216]
end
```

5. Ganti `0.9` menjadi `0.5` dan simpan ke vektor lain. Dengan koefisien lebih kecil, guncangan masa lalu memudar lebih cepat.

Jawaban latihan `while`

1. `i = 27`, 3 pass: $1 \to 3 \to 9 \to 27$, lalu `27 < 20` salah.
2. `i = -2`, setelah 4 pass: $10 \to 7 \to 4 \to 1 \to -2$. Loop baru berhenti setelah kondisi salah, sehingga nilai akhir bisa melewati 0.
3. 0 pass. `5 < 5` sudah salah sebelum pass pertama.
4. Infinite loop: `n` tidak pernah berubah. Perbaikan:

```
n = 1;
while n < 10
    disp(n)
    n = n + 1;
end
```